

# Form, GET e POST

---

## Form, GET e POST

*Come ricevere dati inviati da un form HTML in PHP.*

## Cosa fa un form

Un form HTML permette a una persona di inserire dati e inviarli al server. PHP può leggere quei dati e usarli.

## Un form con GET

```
<form action="cerca.php" method="get">
  <input type="text" name="q">
  <button type="submit">Cerca</button>
</form>
```

`action` indica quale file riceve i dati. `method` indica come vengono inviati.

In `cerca.php` puoi leggere il campo così:

```
<?php
$ricerca = $_GET["q"] ?? "";
echo "Hai cercato: " . htmlspecialchars($ricerca, ENT_QUOTES, 'UTF-8');
```

`$_GET` contiene i dati inviati con metodo GET. `?? ""` evita errori se il campo non esiste.

## Quando usare GET

GET mette i dati nell'indirizzo della pagina. Va bene per ricerche e filtri.

Esempio:

```
cerca.php?q=php
```

Non usarlo per password o dati sensibili.

## Un form con POST

```
<form action="contatto.php" method="post">
  <input type="text" name="nome">
  <textarea name="messaggio"></textarea>
  <button type="submit">Invia</button>
</form>
```

In `contatto.php` :

```
<?php
$nome = $_POST["nome"] ?? "";
$messaggio = $_POST["messaggio"] ?? "";

echo "Grazie, " . htmlspecialchars($nome, ENT_QUOTES, 'UTF-8');
```

`$_POST` contiene i dati inviati con metodo POST.

## Validare sempre

Non fidarti mai dei dati ricevuti da un form. Controlla che ci siano, che abbiano il formato giusto e che non siano troppo lunghi.

```
<?php
if ($nome === "") {
    echo "Il nome e obbligatorio";
}
```

La validazione rende il programma piu sicuro e piu gentile con chi lo usa.

# Mantenere il valore dopo l'invio

Quando un form contiene errori, è gentile mostrare di nuovo i valori inseriti.

```
<?php
$nome = $_POST["nome"] ?? "";
?>

<form method="post">
    <input type="text" name="nome" value="<?= htmlspecialchars($nome,
ENT_QUOTES, 'UTF-8') ?>">
    <button type="submit">Invia</button>
</form>
```

Se il form viene inviato, `$nome` contiene il valore ricevuto. `htmlspecialchars` lo rende sicuro prima di rimetterlo nell'HTML.

# Controllare il metodo della richiesta

Una pagina può essere aperta normalmente o ricevere un invio dal form. Puoi distinguere i due casi con `$_SERVER["REQUEST_METHOD"]`.

```
<?php
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $nome = $_POST["nome"] ?? "";

    if ($nome === "") {
        echo "Il nome è obbligatorio";
    }
}
```

In questo modo validi i dati solo quando il form è stato inviato.

# Regola pratica

Usa GET quando il risultato può stare nell'indirizzo, come una ricerca. Usa POST quando stai inviando dati che cambiano qualcosa o che non vuoi mostrare nell'URL.